

API Endpoints for Multi-Tenant CRM SaaS Platform

This API uses RESTful JSON endpoints with token-based auth (e.g. JWT/OAuth2). Every request carries a tenant (company) context, typically via a path parameter or token claim ¹. Endpoints support pagination (e.g. `?limit` / `?offset`) and filtering (e.g. by status or date) to limit response size ² ³. Below are the required endpoints, organized by category:

1. Auth & Session

Endpoints for user authentication, session management, and obtaining the current user. Include the tenant (company) identifier if needed (e.g. subdomain or a "companyId" field) ¹.

- **POST /auth/login**

Purpose: Authenticate a user (employee or standalone) and issue an access token (and refresh token).

Required body: `{ "email": "<email>", "password": "<password>", (optional) "companyId": "<tenant ID>" }` (or tenant inferred from domain).

Response (200): JSON with `{ "access_token": "<jwt>", "refresh_token": "<token>", "user": { "id", "name", "email", "roles": ["CEO"|"company_admin"|"admin"|"employee"], "companyId" } }`.

- **POST /auth/logout**

Purpose: Invalidate the current access (and refresh) token (log out).

Required: Authorization header with Bearer token.

Response (200): `{ "message": "Logged out successfully" }` or empty body.

- **POST /auth/refresh**

Purpose: Obtain a new access token using a refresh token.

Required body: `{ "refresh_token": "<currentRefreshToken>" }`.

Response (200): New `{ "access_token": "<jwt>", "refresh_token": "<newToken>" }`.

- **GET /auth/me**

Purpose: Get the currently authenticated user's profile.

Required: Bearer token in header.

Response (200): `{ "id", "name", "email", "roles", "companyId", ... }` (user fields).

- **POST /auth/register**

Purpose: Register a new standalone user (who isn't a company employee).

Required body: `{ "name", "email", "password" }`.

Response (201): Created user and tokens `{ "access_token", "refresh_token", "user": { ... } }`.

2. Company Management

Endpoints for creating and managing companies (tenants). Companies have a type (e.g. "client" or "vendor") and their own admins. List endpoints support pagination and filtering by fields such as type, status, or creation date ² ³.

- **GET /companies**

Purpose: List all companies (for platform admin) or search/filter companies.

Query parameters (optional): `?type=client|vendor`, `?status=active|inactive`, `?name=<string>`, and pagination `?limit={n}&offset={m}` ².

Response (200): `{ companies: [ { "id", "name", "type", "status", "created_at", ... }, ... ], total, limit, offset }`.

- **GET /companies/{companyId}**

Purpose: Get detailed info for one company.

Response (200): `{ "id", "name", "type", "address", "contactEmail", "status", "created_at", ... }`.

- **POST /companies**

Purpose: Create a new company (tenant). Used for company self-registration or admin setup.

Required body: `{ "name", "type": "client|vendor", "contactEmail", ... }`.

Optionally include first admin user info or create admin via separate endpoint.

Response (201): Created company `{ "id", "name", "type", "status": "pending|active", ... }`.

- **PUT /companies/{companyId}**

Purpose: Update company details (name, address, contact info, type, etc.). Usually restricted to company_admin or platform admin.

Required body: Fields to update (e.g. `{ "name": "...", "address": "...", "type": "vendor", ... }`).

Response (200): Updated company record.

- **DELETE /companies/{companyId}**

Purpose: Delete or deactivate a company (soft-delete). Typically platform admin only.

Response (204): No content.

3. Employee Management

Endpoints to manage company employees (users within a tenant). These are scoped under a company (`companyId` path). Each employee has a role (CEO, company_admin, admin, employee). List endpoints support filters like role or active status.

- **GET /companies/{companyId}/employees**

Purpose: List all employees of a company.

Query (optional): `?role=CEO|company_admin|admin|employee`, `?status=active|inactive`, plus pagination `?limit&offset` ².

Response (200): { employees: [{ "id", "name", "email", "role", "status", "joined_at", ... }, ...], total }.

- **GET /companies/{companyId}/employees/{employeeId}**

Purpose: Get details of one employee.

Response (200): { "id", "name", "email", "role", "status", "companyId", ... }.

- **POST /companies/{companyId}/employees**

Purpose: Add a new employee to a company (invitation or signup). Requires company_admin/CEO permission.

Required body: { "name", "email", "password", "role": "...", ... }.

Response (201): Created user { "id", "name", "email", "role", ... }.

- **PUT /companies/{companyId}/employees/{employeeId}**

Purpose: Update an employee's information (name, email, role, status, etc.). Company_admin or higher.

Required body: Fields to update (e.g. { "role": "admin", "status": "inactive" }).

Response (200): Updated employee record.

- **DELETE /companies/{companyId}/employees/{employeeId}**

Purpose: Remove (or deactivate) an employee from the company. Company_admin/CEO only.

Response (204): No content.

4. Project & Proposal Management

Projects and proposals use the same "projects" resource with a `status` field (e.g. `proposed`, `active`, `completed`, `rejected`). Client companies post proposals which vendor companies can accept. Endpoints allow filtering by status, client company, or vendor company.

- **GET /projects**

Purpose: List all projects/proposals visible to the user. Employees of a client see their company's proposals and projects, vendor employees see proposals assigned to them, admins see all.

Query (optional): `?status=proposed|active|completed`, `?clientId={companyId}`, `?vendorId={companyId}`, plus pagination `?limit&offset` ².

Response (200): { projects: [{ "id", "title", "description", "status", "clientCompanyId", "vendorCompanyId", "created_at", ... }, ...], total }.

- **GET /projects/{projectId}**

Purpose: Get details of a specific project/proposal.

Response (200): { "id", "title", "description", "status", "clientCompanyId", "vendorCompanyId", "budget", "startDate", "endDate", ... }.

- **POST /projects**

Purpose: Create a new project proposal (by a client company).

Required body: { "title", "description", "clientCompanyId", "vendorCompanyId",

"budget", ... }]. The initial `status` is set to `proposed`.

Response (201): Created project proposal { "id", "status": "proposed", ... }.

- **PATCH /projects/{projectId}**

Purpose: Update a project or change its status (e.g. vendor accepts or rejects).

Required body: Fields to update; for example, to accept a proposal the vendor might set { "status": "active", "startDate": "...", ... }; to reject { "status": "rejected" }. Clients may update details while still proposed.

Response (200): Updated project record.

- **DELETE /projects/{projectId}**

Purpose: Cancel a proposal or remove a project. Clients can cancel their proposals; platform admins can delete any.

Response (204): No content.

5. Review System

Standalone users post reviews about companies. Reviews belong to a company and have fields like rating and comment. Employees cannot review their own company; only neutral users do.

- **GET /companies/{companyId}/reviews**

Purpose: List all reviews for a company.

Query (optional): ?rating=1..5, ?dateFrom=<ISO>, ?dateTo=<ISO>, plus pagination.

Response (200): { reviews: [{ "id", "userId", "rating", "title", "comment", "created_at" }, ...], total }.

- **POST /companies/{companyId}/reviews**

Purpose: Submit a new review for a company. Only standalone users (not company employees) use this.

Required body: { "rating": 1..5, "title": "string", "comment": "text" }.

Response (201): Created review { "id", "userId", "companyId", "rating", "comment", "created_at", ... }.

- **GET /companies/{companyId}/reviews/{reviewId}**

Purpose: Get one review's details.

Response (200): { "id", "userId", "companyId", "rating", "title", "comment", "created_at", ... }.

- **PUT /companies/{companyId}/reviews/{reviewId}**

Purpose: Edit an existing review (only the author or an admin).

Required body: Updated fields (rating, comment, etc.).

Response (200): Updated review object.

- **DELETE /companies/{companyId}/reviews/{reviewId}**

Purpose: Delete a review (by author or admin).

Response (204): No content.

- **GET /users/{userId}/reviews**

Purpose: (Optional) List all reviews written by a given user.

Query: `?companyId=` etc.

Response (200): `{ reviews: [ ... ], total }`.

6. Support Query System

Supports “ticket” style queries from any user (standalone or employee). Similar to helpdesk tickets (see examples from Zendesk: GET list, POST create, etc. ⁴). Each support query has fields like subject, message, status, and is linked to a creator (user or employee) and optionally a company.

- **GET /support-queries**

Purpose: List support queries visible to the user. Employees see queries they opened or company-wide queries; standalone users see their own. Platform admins see all.

Query (optional): `?status=open|closed|pending`, `?companyId=`, `?userId=`, plus pagination.

Response (200): `{ queries: [ { "id", "subject", "status", "createdBy", "companyId?", "created_at", ... }, ... ], total }`.

- **GET /support-queries/{queryId}**

Purpose: Get details of a specific support query.

Response (200): `{ "id", "subject", "message", "status", "createdBy", "companyId?", "created_at", "updated_at", ... }`.

- **POST /support-queries**

Purpose: Submit a new support query. Both standalone users and company employees use this.

Required body: `{ "subject", "message", (optional) "companyId" }`. If an employee submits, their `companyId` is implied.

Response (201): Created query `{ "id", "status": "open", "createdBy", "companyId?", "created_at", ... }`.

- **PUT /support-queries/{queryId}**

Purpose: Update a support query (e.g. add an internal note, change status to closed). Accessible to support agents or admins.

Required body: e.g. `{ "status": "closed", "response": "Answer or resolution text" }`.

Response (200): Updated query object.

- **DELETE /support-queries/{queryId}**

Purpose: (Optional) Delete or close a support query. Usually only admin.

Response (204): No content.

7. Platform Admin Tools (Global)

Endpoints for platform administrators to view/manage all data across tenants. These are typically prefixed with `/admin`. Admins have unrestricted access to all companies, users, projects, etc.

- **GET /admin/companies** – List **all** companies (with pagination/filters).
- **GET /admin/companies/{companyId}** – Get any company's details.
- **PUT /admin/companies/{companyId}** – Update any company (e.g. change status).
- **DELETE /admin/companies/{companyId}** – Permanently remove or deactivate a company.
- **GET /admin/employees** – List all employees across all companies. Can filter by companyId or role.
- **GET /admin/employees/{employeeId}** – Get details of any employee.
- **PUT /admin/employees/{employeeId}** – Update any employee's account.
- **DELETE /admin/employees/{employeeId}** – Remove an employee.
- **GET /admin/users** – List all standalone users.
- **GET /admin/users/{userId}** – Get any user's details.
- **PUT /admin/users/{userId}** – Update a user's info or status.
- **DELETE /admin/users/{userId}** – Delete a user.
- **GET /admin/projects** – List all projects/proposals across the system. Filter by status/client/vendor.
- **GET /admin/projects/{projectId}** – Details of any project.
- **PUT /admin/projects/{projectId}** – Update any project.
- **DELETE /admin/projects/{projectId}** – Delete any project.
- **GET /admin/reviews** – List all company reviews in the system.
- **GET /admin/reviews/{reviewId}** – Get any review.
- **DELETE /admin/reviews/{reviewId}** – Remove a review.
- **GET /admin/support-queries** – List all support queries. Filter by status or user.
- **GET /admin/support-queries/{queryId}** – View any query.
- **PUT /admin/support-queries/{queryId}** – Update/close any query.
- **DELETE /admin/support-queries/{queryId}** – Delete a query.

Each endpoint above uses JSON for requests and responses. Responses typically include the resource's primary fields (e.g. `id`, names, status flags, timestamps). For list endpoints, the returned JSON contains an array of objects and possibly metadata (`total`, `limit`, etc.). All query/list endpoints support pagination

(`limit`, `offset`) and relevant filters ² ³. These designs follow RESTful best practices (e.g. Zendesk-style ticket CRUD ⁴ and Azure API guidelines ² ³).

Sources: Design patterns from REST API best practices ² ³ ¹ and example ticketing APIs ⁴, adapted to a multi-tenant CRM context.

¹ ² ³ Web API Design Best Practices - Azure Architecture Center | Microsoft Learn

<https://learn.microsoft.com/en-us/azure/architecture/best-practices/api-design>

⁴ Zendesk - Relevance AI Documentation

<https://relevanceai.mintlify.app/integrations/popular-integrations/zendesk>